



Figure 3: Internal architecture of the Spark engine

and structured datasets, is built on top of this. Spark Streaming allows live streaming and analysis of data loaded into RDDs as mini-batches. MLlib is an extensive library, which helps to implement machine learning methods on Big Data sets. It is created by a community of programmers from across the world. GraphX is a distributed graph-processing framework, which provides an API for expressing graph computation that can model the user-defined graphs by using the Pregel abstraction API. In addition, GraphX includes a growing collection of graph algorithms and builders to optimise graph analytics tasks.

Spark applications run independently on sets of clusters that are managed by a SparkContext object in the driver program. A SparkContext instance can connect with managers such as Mesos or YARN and can allocate resources to different commodity machines for optimised performance. After allocation, the executors on each job receive the application code and tasks, which are utilised to execute the job. Each Spark application has its own executors which can do multi-threading. Data needs to be stored on external storages for different Spark applications to share it.

Getting started with the Apache Spark engine

The following section explores how to start the Spark engine and get the services started. It will show how to execute existing programs, how to start the client or server and how to launch the shell.

Starting Spark services and the shell

We will change the directory, go into Spark's folder and manually start the master cluster by using the following command:

```
cd spark-2.0.0-bin-hadoop2.6
./sbin/start-master.sh
```

After running this, you can view the user interface of the master node by typing the following command in the browser:

```
http://localhost:8080
```

You can start the slave node by using the following command:

```
./sbin/start-slave.sh <name of slave node to run>
```

To check if nodes are running, execute the following:

```
jps
```

Running the Spark shell

You can run the Spark shell for Scala using the command given below:

```
$ bin/spark-shell
```

You can run the Spark shell for Python by using the following command:

```
$ bin/pyspark
```

Submitting an existing application in Spark

First, let us compile a file that contains the code for a program which is to be run in Spark later on:

```
$ scalac -classpath "spark-core_2.10-2.0.0.jar:/usr/local/spark/lib/spark-assembly-2.0.0-hadoop2.6.0.jar" <file name>
```

Then, let's create a JAR file out of the compiled file, as follows:

```
jar -cvf wordcount.jar SparkwordCount*.class spark-core_2.10-1.3.0.jar/usr/local/spark/lib/spark-assembly-1.4.0-hadoop2.6.0.jar
```

Now, submit the JAR file to Spark to run the application, as follows:

```
$ spark-submit --class <application name> --master local <jar file name>
```

Writing and executing basic scripts in the Apache Spark engine

Since we have already learnt how to start the shell and submit jobs through it after creating and compiling JAR files, let's now write and execute a simple WordCount example in Scala to be deployed on Spark.

First, create a simple *input.txt* file from the sentence given below and put it in the Spark application folder containing all other jar files and program code:

"This is my first small word count program using Spark. I will use a simple MapReduce program made in Scala to count the frequency of each word."

Next, open the Spark shell:

```
$ spark-shell
```

Then make an RDD, which will read the data from our *input.txt* file. *sc* is SparkContext object, which is a manager